

IMPLEMENTING AN INTEGRATED DEVELOPMENT ENVIRONMENT: A QUICK START GUIDE FOR PL RESEARCHERS

PLDI 2026 Tutorial — 15th of June, 2026

Bergen Language Design Laboratory, University of Bergen, Norway

Mikhail Barash

- Associate Professor, Bergen Language Design Laboratory (University of Bergen, Norway).
- Ph.D. on formal grammars and parsing (2015).
- Language workbenches, programming language specification, standardization, and evolution.

Alberto Tontoni

- Ph.D. student, Bergen Language Design Laboratory.
- IDEs, programming language specification.

AGENDA

- **SLIDES** IDEs, textual language workbenches.
- **SLIDES** Running example: *Bean* (PLDI 2025).
- **HANDS-ON** **Langium** (VSCode).
- **SLIDES** Tool-first languages and projectional editing.
- **HANDS-ON** **JetBrains MPS**.

M. Barash, V. Pech. Teaching MPS: Experiences from Industry and Academia. DSLs in Practice 2021.

TOOLS TO FOLLOW THE TUTORIAL

- **Langium:** playground (web-based, no installation).
- **Langium:** VSCode extension (installation required).
- **MPS:** installation required.

➤ github.com/BLDL/PLDI2026-tutorial-IDE

IDE FEATURES

- syntax-aware editor
- syntax highlighting
- background validation
- incremental syntax checking
- content assist (autocompletion)
- hyperlinking and hovering
- quickfixes
- autoformatting

LANGUAGE WORKBENCHES

“ *Language workbenches are tools to define software languages together with tailored Integrated Development Environments for them.* ”

“ Language workbenches are tools to define software languages together with tailored Integrated Development Environments for them.

User roles:

- language user
- **language engineer**
- meta-language engineer

RUNNING EXAMPLE: *BEAN*

- Ariel E. Kellison, Laura Zielinski, David Bindel, Justin Hsu.
2025. **Bean: A Language for Backward Error Analysis.** Proc.
ACM Program. Lang. 9, PLDI, Article 221 (June 2025), 25
pages.
- backward error analysis, stability of numerical programs
 - linear, discrete, unit variables
 - bindings
 - pair constructors
 - pattern matching for destructors
 - primitive arithmetic operations
 - injections
 - case analysis

```

1  {}
2  {(A : ((num, num), (num, num))) (b : (num, num))}
3
4  /*
5      Computes x where Ax = b, A is a 2x2 lower triangular matrix and
6      b and x are vectors in R^2.
7  */
8
9  let (a0, a1) = A;
10 let (a00, a01) = a0;
11 let (a10, a11) = a1;
12 let (b0, b1) = b;
13
14 let x0_or_err = div b0 a00;
15 case x0_or_err of {
16   inl (x0) =>
17     dlet d_x0 = !x0;
18     let s0 = dmul d_x0 a10;
19     let s1 = sub b1 s0;
20     let x1_or_err = div s1 a11;
21     case x1_or_err of {
22       inl (x1) => inl () (d_x0, x1)
23       | inr (err) => inr (dnum, num) err
24     }
25   | inr (err) => inr (dnum, num) err
26 }

```

<https://github.com/Athena-Types/Bean/blob/main/examples/LinSolve.be>

$$\begin{aligned}
\sigma, \tau &::= \mathbf{unit} \mid \mathbf{num} \mid \sigma \otimes \sigma \mid \sigma + \sigma \mid \alpha && \text{(types)} \\
\alpha &::= m(\sigma) && \text{(discrete types)} \\
\Gamma &::= \emptyset \mid \Gamma, x :_r \sigma && \text{(linear typing contexts)} \\
\Phi &::= \emptyset \mid \Phi, z : \alpha && \text{(discrete typing contexts)} \\
e, f &::= x \mid z \mid () \mid !e \mid (e, f) \mid \mathbf{inl} \, e \mid \mathbf{inr} \, e \\
&\quad \mid \mathbf{let} \, x = e \mathbf{in} \, f \mid \mathbf{let} \, (x, y) = e \mathbf{in} \, f \mid \mathbf{dlet} \, z = e \mathbf{in} \, f \mid \mathbf{dlet} \, (x, y) = e \mathbf{in} \, f \\
&\quad \mid \mathbf{case} \, e' \mathbf{of} \, (\mathbf{inl} \, x.e \mid \mathbf{inr} \, y.f) \mid \mathbf{add} \, e \, f \mid \mathbf{sub} \, e \, f \mid \mathbf{mul} \, e \, f \mid \mathbf{dmul} \, e \, f \mid \mathbf{div} \, e \, f \\
&&& \text{(expressions)}
\end{aligned}$$
Fig. 2. Grammar for **BEAN** types and terms.



DT ::=	DISCRETE TYPES
dnum	discrete numeric type
(DT, DT)	discrete tensor product
DT + DT	discrete sum type
T ::=	TYPES
DT	discrete types
()	single-valued unit type
num	numeric type
(T, T)	tensor product
T + T	sum type
v, w ::=	VALUES
()	value of unit type
a	variables
(v, w)	tensor pairs
inl v	injection into sum
inr v	injection into sum
!x	!-constructor, where x is linear
e, f ::=	EXPRESSIONS
v	values
let (x, y) = v; e	linear tensor destructor
dlet (x, y) = v; e	discrete tensor destructor
case v {inl x => e inr x => f}	case analysis
let x = e; f	let-binding, where e is linear
let x : T = e; f	let-binding with type annotation
dlet x = e; f	let-binding, where e is discrete
dlet x : DT = e; f	let-binding with type annotation
op a b	op in (add, mul, sub, div, dmul), a and b are variables

Grammars

$$G = (\Sigma, N, R, S)$$

- $A \rightarrow \alpha$, with $\alpha \in (\Sigma \cup N)^*$

Grammars

$$G = (\Sigma, N, R, S)$$

- $A \rightarrow \alpha$, with $\alpha \in (\Sigma \cup N \cup \{\text{ID}\})^*$

Grammars

$$G = (\Sigma, N, R, S)$$

- $A \rightarrow \alpha$, with $\alpha \in (\Sigma \cup N \cup \{\mathbf{ID}\} \cup \mathcal{X})^*$

Grammars

$$G = (\Sigma, N, R, S)$$

- $A \rightarrow \alpha$, with $\alpha \in (\Sigma \cup N \cup \{\mathbf{ID}\} \cup \mathcal{X})^*$
- cross-references $\mathcal{X} = \{\mathbf{ID} \hookrightarrow A \mid A \in N\}$

Grammars

$$G = (\Sigma, N, R, S)$$

- $A \rightarrow \alpha$, with $\alpha \in (\Sigma \cup N \cup \{\mathbf{ID}\} \cup \mathcal{X})^*$
- cross-references $\mathcal{X} = \{\mathbf{ID}_{\hookrightarrow A} \mid A \in N\}$
 - $\mathbf{ID}_{\hookrightarrow A}$: an identifier that is a cross-reference to an identifier declared in a substring derived from nonterminal A

Grammars

$$G = (\Sigma, N, R, S)$$

- $A \rightarrow \alpha$, with $\alpha \in (\Sigma \cup N \cup \{\mathbf{ID}\} \cup \mathcal{X})^*$
- cross-references $\mathcal{X} = \{\mathbf{ID} \hookrightarrow A \mid A \in N\}$
 - $\mathbf{ID} \hookrightarrow A$: an identifier that is a cross-reference to an identifier declared in a substring derived from nonterminal A

Langium:

- adaptive $\text{LL}(\ast)$ parsing algorithm; subfamily of CFG
- output: VSCode extension + Language Server Protocol
- configurable styling, code services, ...

T. Parr, S. Harwell, K. Fisher. Adaptive $\text{LL}(\ast)$ parsing: the power of dynamic analysis. OOPSLA 2014.

Language workbench:

- base **metalanguages**: $B_1, \dots, B_k$
- user **languages**: $L_1, \dots, L_n$

Language workbench:

- base **metalanguages**: $B_1, \dots, B_k$

$$B_i = f_i(B_1, \dots, \widehat{B_i}, \dots, B_k), \quad (B_1, \dots, B_k) = \text{lfp}(F)$$

- user **languages**: $L_1, \dots, L_n$

Language workbench:

- base **metalanguages**: $B_1, \dots, B_k$

$$B_i = f_i(B_1, \dots, \widehat{B_i}, \dots, B_k), \quad (B_1, \dots, B_k) = \text{lfp}(F)$$

- user **languages**: $L_1, \dots, L_n$

$$L_i = g_i(B_1, \dots, B_k, L_{j_1}, \dots, L_{j_\ell}, \widehat{L_i})$$

Language workbench:

- base **metalanguages**: $B_1, \dots, B_k$

$$B_i = f_i(B_1, \dots, \widehat{B_i}, \dots, B_k), \quad (B_1, \dots, B_k) = \text{lfp}(F)$$

- user **languages**: $L_1, \dots, L_n$

$$L_i = g_i(B_1, \dots, B_k, L_{j_1}, \dots, L_{j_\ell}, \widehat{L_i})$$

“Language-Aspects-Concepts” idiom:

- $L = \{C_1, \dots, C_m\}$
- $C_i = \langle C_i |_{A_1}, \dots, C_i |_{A_\ell} \rangle$
- $A_i = \langle \mathcal{S}, \mathcal{F} \rangle$

M. Barash. The Next 700 Language Workbenches. SLE 2021.

S. Dmitriev. Language-oriented programming: The Next Programming Paradigm. 2004.

Language workbench:

- base **metalanguages**: $B_1, \dots, B_k$

$$B_i = f_i(B_1, \dots, \widehat{B_i}, \dots, B_k), \quad (B_1, \dots, B_k) = \text{lfp}(F)$$

- user **languages**: $L_1, \dots, L_n$

$$L_i = g_i(B_1, \dots, B_k, L_{j_1}, \dots, L_{j_\ell}, \widehat{L_i})$$

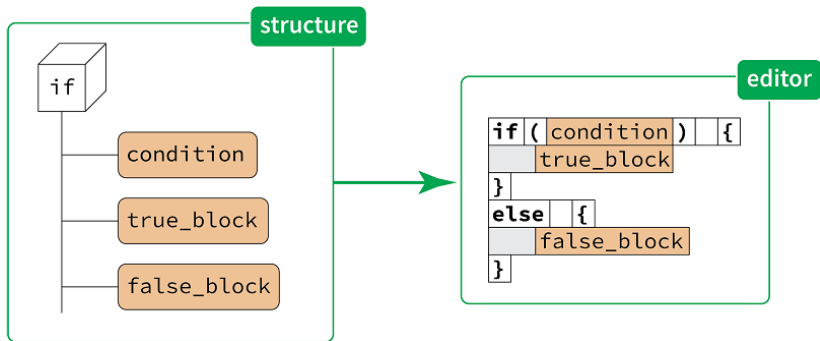
“Language-Aspects-Concepts” idiom:

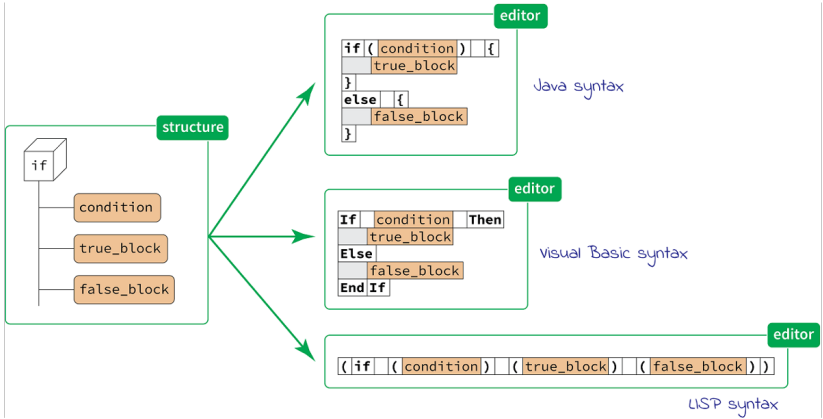
- $L = \{C_1, \dots, C_m\}$
- $C_i = \langle C_i |_{A_1}, \dots, C_i |_{A_\ell} \rangle$
- $A_i = \langle \mathcal{S}, \mathcal{F} \rangle$
- $A_i \longleftrightarrow B_i$

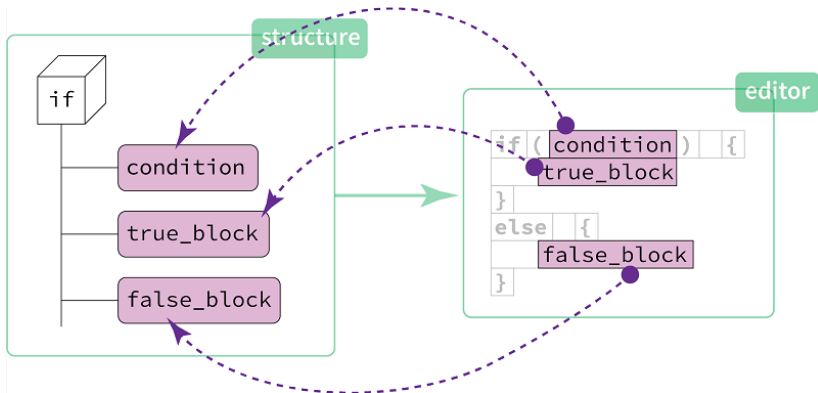
M. Barash. The Next 700 Language Workbenches. SLE 2021.

S. Dmitriev. Language-oriented programming: The Next Programming Paradigm. 2004.

- **STRUCTURE:** abstract syntax
- **EDITOR:** concrete syntaxes
- **INTENTIONS:** quickfixes
- **BEHAVIOUR:** private methods
- **CONSTRAINTS:** restrictions on aggregations
- **TYPESYSTEM:** specialized validations
- **GENERATOR:** code transformation







$C ::=$

$C ::=$

$\overrightarrow{[c_1 \cdots c_\ell]}$

horizontal collection

CELL-BASED PROJECTIONS

$C ::=$

$\overrightarrow{[c_1 \cdots c_\ell]}$

horizontal collection

$|\downarrow [c_1 \cdots c_\ell] \uparrow$

vertical collection

CELL-BASED PROJECTIONS

$C ::=$

$\overrightarrow{[c_1 \cdots c_\ell]}$

horizontal collection

$|\downarrow [c_1 \cdots c_\ell] \uparrow$

vertical collection

$|\text{"}t\text{"}$

constant value

CELL-BASED PROJECTIONS

$C ::=$

$\overrightarrow{[c_1 \cdots c_\ell]}$

horizontal collection

$\downarrow [c_1 \cdots c_\ell] \uparrow$

vertical collection

$"t"$

constant value

v

computed value

CELL-BASED PROJECTIONS

$C ::=$

$\overrightarrow{[c_1 \cdots c_\ell]}$

horizontal collection

$\downarrow [c_1 \cdots c_\ell] \uparrow$

vertical collection

$"t"$

constant value

v

computed value

$\bar{r}$

reference ($\bar{r} \in \Omega_{\bar{r}}$)

CELL-BASED PROJECTIONS

$C ::=$

$\overrightarrow{[c_1 \cdots c_\ell]}$

horizontal collection

$|\downarrow [c_1 \cdots c_\ell] \uparrow$

vertical collection

$|\text{"}t\text{"}$

constant value

$|\nu$

computed value

$|\bar{r}$

reference ($\bar{r} \in \Omega_{\bar{r}}$)

$|\mathbf{s}[C]$

computed cell

CELL-BASED PROJECTIONS

$C ::=$

$\overrightarrow{[c_1 \cdots c_\ell]}$

horizontal collection

| $\downarrow [c_1 \cdots c_\ell] \uparrow$

vertical collection

| $"t"$

constant value

| v

computed value

| $\bar{r}$

reference ($\bar{r} \in \Omega_{\bar{r}}$)

| $_s[C]$

computed cell

| $[C]_f$

formatting directive

EXAMPLE

projection π_1 of “if statement”

$\downarrow[$

$\overrightarrow{[}$ `["if"]bold` `("v1")` `["then"]bold` $\overleftarrow{]}$

$\downarrow[$ `v2` $\uparrow]$ `indent`

$\overrightarrow{[}$ `["end"]bold` `["then"]bold` $\overleftarrow{]}$

`v3≠null` $[[$

$\downarrow[$

`["else"]bold`

$\downarrow[$ `v3` $\uparrow]$ `indent`

$\overrightarrow{[}$ `["end"]bold` `["else"]bold` $\overleftarrow{]}$

$\uparrow]$

$]]$

$\uparrow]$

SIDE TRANSFORMATIONS

$$\tau : N \times C \times \{\text{LEFT}, \text{RIGHT}\} \times \text{String} \times \Gamma \rightarrow N$$

- cell $c \in C$ in a projection $\pi(n)$ of a node $n \in N$
- trigger $\alpha \in \text{String}$
- condition $\gamma \in \Gamma$
- new node $n' \in N$ that replaces n

SIDE TRANSFORMATIONS

$\tau : N \times C \times \{\text{LEFT}, \text{RIGHT}\} \times \text{String} \times \Gamma \rightarrow N$

- cell $c \in C$ in a projection $\pi(n)$ of a node $n \in N$
- trigger $\alpha \in \text{String}$
- condition $\gamma \in \Gamma$
- new node $n' \in N$ that replaces n

Example: if statement

- original node: $n = \langle v_1, v_2, \text{null} \rangle$
- original projection: $\pi_1(n)$
- transformation: $(\text{"then"} \mid_8, \text{RIGHT}, \text{"else"}, \text{true})$
- new node: $n' = \langle v_1, v_2, v_3 \rangle$, with $v_3 \neq \text{null}$

Validation

$$\kappa : N \times \Gamma \mapsto N \times M$$

- attach diagnostic message $\mu \in M$ to node $n' \in N$ if condition $\gamma \in \Gamma$ holds for node n

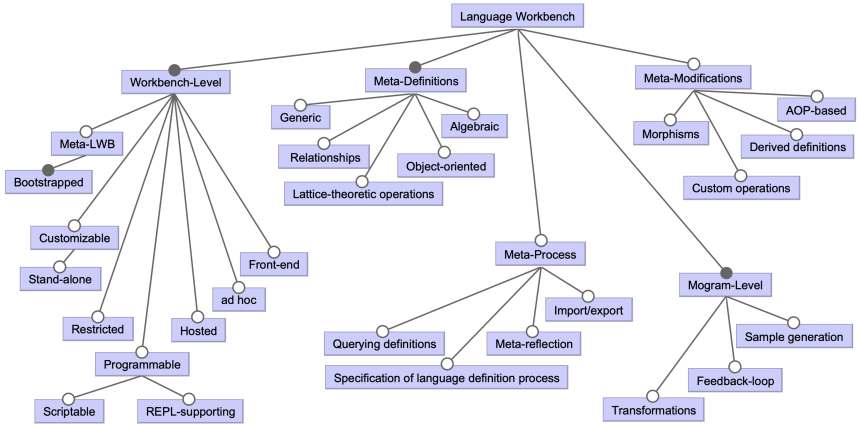
Quickfix

$$\tau : N \times M \times \Gamma \mapsto N$$

- replace node $n \in N$ with node $n' \in N$ if n has a diagnostic message $\mu \in M$ and condition $\gamma \in \Gamma$ holds for n

- S. Erdweg et al. Evaluating and comparing language workbenches: Existing results and benchmarks for the future. *Comput. Lang. Syst. Struct.* 44. 2015.
- E. Visser. Declarative Language Definition with Spoofax. **PLDI 2020 Tutorial.**
- E. Visser. Declare Your Language (Part 1): Hands-On Spoofax Tutorial. **POPL 2016 Tutorial.**
- E. Visser. Declare Your Language (Part 2): Name Binding with Scope Graphs. **POPL 2016 Tutorial.**

LANGUAGE WORKBENCHES



M. Barash. The Next 700 Language Workbenches. SLE 2021.